DemandLens

Turning fragmented retail signals into timely, explainable action

TRACK	02 — Customers, Channels & Demand
SOLUTION	Customer identity • Inventory risk • Returns margin • Churn signals
DELIVERABLE	Hackathon MVP, source code, synthetic datasets and deployment guide
TECH STACK	Python • Streamlit • Pandas • Plotly • Pytest
LICENSE	MIT

Project report and implementation guide

This document describes the prototype, its assumptions, how to run it, and how it can be validated and extended. All bundled data is synthetic and for demonstration only.

1. Executive Summary

DemandLens is a lightweight decision-support workspace for retail, e-commerce, direct-to-consumer (D2C), and quick-commerce teams. It demonstrates how fragmented customer profiles, inventory levels, order returns, and support tickets can be transformed into an actionable operational view.

The prototype combines four workflows: potential duplicate customer detection, inventory days-of-cover risk, estimated return-related gross-margin exposure, and support-ticket churn-signal triage. A Streamlit interface exposes these workflows, while CSV download buttons let a user export review lists.

The MVP deliberately uses transparent rules and synthetic records. It is designed to demonstrate a credible end-to-end workflow rather than claim production-grade forecasting or validated churn prediction.

2. Problem Statement

Omnichannel retailers commonly maintain customer and operational data in separate systems. The same customer may appear under different channel-specific identifiers. Inventory risk may only become visible after a product is unavailable. Return rates can conceal gross-margin erosion, and support messages can contain cancellation or dissatisfaction clues that are not routed to retention workflows.

Target users

- Growth, CRM, and customer-retention teams that need a consolidated customer view.
- Inventory planners and merchandisers who need earlier warning of low stock cover.
- Category managers and finance teams investigating return-related margin exposure.
- Customer-experience teams prioritizing tickets that may require urgent recovery.

Why the problem matters

Fragmented data creates manual reconciliation work, delayed interventions, avoidable lost sales, and missed opportunities to recover unhappy customers. Teams need not only a chart but a transparent reason for each alert and a clear next action.

Problem statement

How might we help retail and D2C teams combine customer, inventory, returns, and support signals into one explainable workspace that prioritizes operational actions before small issues become lost sales or customer churn?

3. Proposed Solution

DemandLens offers a compact decision-support layer with four modules. The current implementation reads local CSV files, applies explainable heuristics, and displays results in a browser-based dashboard.

3.1 Customer 360

Email addresses are trimmed and lowercased, while phone numbers are normalized to digits and compared using the final ten digits. Records sharing an exact normalized email or phone are flagged as possible duplicates. The system does not automatically merge profiles; a human should review each candidate.

3.2 Inventory Risk

Estimated days of cover is calculated as stock on hand divided by average daily demand. A configurable threshold labels products as high, medium, or low risk. Each label maps to a suggested action, such as checking nearby stock or reviewing supplier lead time.

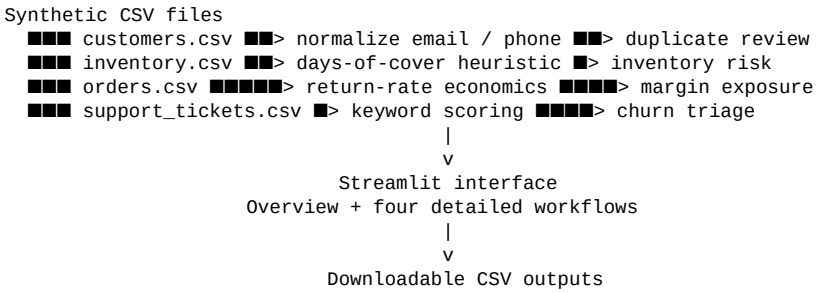
3.3 Returns & Margin

The prototype estimates returned units from units sold multiplied by return rate. Estimated margin at risk is the returned-unit estimate multiplied by positive unit margin. This simplified view helps identify categories for investigation; it is not an accounting ledger.

3.4 Churn Signals

Support-ticket text is scanned for explicit phrases such as cancel, refund, late, broken, never again, unresolved, and charged twice. Weighted keyword cues create a score capped at ten and a corresponding triage level. The score is a prioritization cue, not a probability that a customer will churn.

4. System Architecture



Technology stack

- Python for the application and analytics functions.
- Pandas for loading, transforming, and exporting tabular data.
- Streamlit for the interactive web interface.
- Plotly for interactive charts.
- Pytest for automated unit tests.

Repository modules

The main entry point is app.py. Reusable calculations live in src/analytics.py. Synthetic input files are stored in data/. Automated checks are in tests/. Setup, architecture, demo narration, deployment, and pitch content are documented in docs/.

5. Functional Requirements and Features

Module	Input	Output / user value
Customer 360	Customer ID, name, email, phone, channel	Normalized contact keys and duplicate review flags
Inventory Risk	Stock on hand, average daily demand	Days of cover, risk level, recommended action
Returns & Margin	Units sold, price, unit cost, return rate	Estimated returned units/value and margin at risk
Churn Signals	Support-ticket text and metadata	Keyword evidence, risk score, triage level, next action

Interface pages

- Overview: high-level KPIs and charts for inventory and return exposure.
- Customer 360: filter to possible duplicate records and export a review CSV.
- Inventory Risk: change the low-cover threshold and export product risk.
- Returns & Margin: inspect estimated exposure by order/product and export analysis.

- Churn Signals: filter ticket risk levels and export a prioritized triage list.

6. Data Design

The repository includes four synthetic CSV files. The records are fabricated for demonstration and do not represent actual customers or commercial results.

customers.csv

Contains customer_id, customer_name, email, phone, and channel. A few intentional exact contact overlaps demonstrate how the duplicate review works.

inventory.csv

Contains product_id, product_name, category, stock_on_hand, avg_daily_demand, and supplier_lead_time_days. The lead-time field is included for future extension; the current risk formula does not yet incorporate it.

orders.csv

Contains order_id, product_name, category, units_sold, selling_price, unit_cost, and return_rate. Return rate is represented as a fraction between zero and one.

support_tickets.csv

Contains ticket_id, customer_id, ticket_text, created_date, and channel. Ticket wording is synthetic and is intended only to demonstrate transparent triage logic.

7. Key Calculations

7.1 Days of cover

```
days_of_cover = stock_on_hand / average_daily_demand
```

If average daily demand is zero or missing, the current implementation treats days of cover as infinite, preventing division-by-zero errors. In production, stale or missing demand should be explicitly flagged.

7.2 Estimated return exposure

```
estimated_returned_units = units_sold × return_rate
unit_margin = selling_price − unit_cost
estimated_margin_at_risk = estimated_returned_units × max(unit_margin, 0)
```

This estimate assumes the product margin associated with returned units is exposed. Actual economics depend on refunds, resale recovery, product condition, reverse logistics, payment fees, and accounting policy.

7.3 Churn signal score

Each configured keyword contributes a fixed weight. Matching terms are shown alongside the score so the user can inspect the evidence. This approach is simple and auditable, but it may miss context, sarcasm, negation, and phrases that do not match the keyword list.

7.4 Identity matching

Contact fields are normalized before exact duplicate checks. Shared family phone numbers, recycled numbers, typos, and common or shared email addresses can produce false matches. Therefore, the tool creates a review list instead of merging records.

8. Setup and Usage

Requirements

Python 3.10 or newer is recommended. Install the dependencies listed in requirements.txt from the repository root.

```
python -m venv .venv
# Windows
.venv\Scripts\activate
# macOS / Linux
source .venv/bin/activate
pip install -r requirements.txt
streamlit run app.py
```

Open the local URL printed in the terminal, usually http://localhost:8501. Use the sidebar to navigate between Overview, Customer 360, Inventory Risk, Returns & Margin, and Churn Signals.

Run automated tests

```
pytest -q
```

The included tests cover duplicate detection, inventory risk labels, return-margin arithmetic, and high-risk ticket scoring.

9. Testing and Evaluation Plan

Unit tests verify key calculations on small controlled examples. Before any real-world use, the system needs evaluation against reviewed examples and business outcomes.

Suggested pilot metrics

- Customer matching: precision of duplicate flags confirmed by trained reviewers.
- Inventory: stock-out frequency, alert lead time, false-alert rate, and missed stock-outs.
- Returns: estimate error compared with actual refunds, recovery, and reverse-logistics costs.
- Churn triage: precision/recall against a labeled set of tickets and subsequent customer outcomes.
- Operations: time to investigate an alert, action completion rate, and user adoption.

A pilot should establish a baseline, define a measurement window, document data-quality gaps, and compare results after the workflow is introduced. Do not claim improved sales or reduced churn until the effect is measured.

10. Privacy, Security, and Responsible Use

- Only synthetic data is included in the public demo.
- Do not commit real names, phone numbers, emails, order histories, support tickets, credentials, or API keys to a public repository.
- A production deployment needs authentication, role-based authorization, secure secrets, encryption, retention limits, and audit logs.
- Human review is required before merging customer profiles or taking consequential customer actions.
- Scores should be explained as heuristic indicators until they have been validated and calibrated on representative data.

11. Limitations

- Inventory calculations do not model seasonality, promotions, supplier lead-time variability, safety stock, or inbound purchase orders.
- Customer identity resolution is based on exact normalized contact matches rather than probabilistic matching.
- Returns economics are simplified and omit recovery value and several operational costs.
- Ticket scoring uses keywords and is not a trained machine-learning model.

- The current prototype reads CSVs and does not connect to live POS, CRM, commerce, warehouse, or helpdesk systems.
- The dashboard is a demonstration and does not include production authentication or multi-user data isolation.

12. Roadmap

- Add connectors to approved commerce, POS, CRM, inventory, and helpdesk sources.
- Add data-quality checks, freshness indicators, and a persistent review queue.
- Develop demand forecasts with backtesting, seasonality, lead-time uncertainty, and safety-stock logic.
- Evaluate probabilistic entity resolution with human-in-the-loop review.
- Train and calibrate churn models using consented, labeled historical outcomes.
- Add access controls, audit trails, monitoring, and deployment security.

13. Business Model Hypothesis

DemandLens could be offered as B2B SaaS for smaller omnichannel retailers and D2C brands. A Starter tier could provide core reporting for one channel; a Growth tier could add multiple channels, alerts, and team workflows; and a Scale tier could include custom connectors, access controls, audit logs, and support commitments. Pricing should be validated through customer interviews and pilot willingness-to-pay rather than assumed in advance.

14. Conclusion

DemandLens demonstrates an end-to-end prototype that combines four common retail pain points in one explainable interface. Its value proposition is to help teams see where to investigate and what action to consider, rather than merely presenting disconnected reports. The next step is a controlled pilot using approved data, measured against explicit operational outcomes.

Appendix A — Repository Contents

```
app.py                          Streamlit application
src/analytics.py                Reusable analytics logic
data/customers.csv              Synthetic customer profiles
data/inventory.csv              Synthetic inventory snapshot
data/orders.csv                 Synthetic sales/returns data
data/support_tickets.csv        Synthetic support-ticket text
tests/test_analytics.py         Unit tests
docs/PROBLEM_STATEMENT.md       Problem and target users
docs/ARCHITECTURE.md            Architecture and heuristics
docs/DEMO_SCRIPT.md             Under-3-minute demo script
docs/DEPLOYMENT.md              Local and Streamlit deployment
docs/PITCH_CONTENT.md           Pitch and business model notes
README.md                       Main project documentation
LICENSE                         MIT licence
pitch_deck.pptx / .pdf          Pitch presentation
```

Appendix B — License

The source project is distributed under the MIT License. See the repository LICENSE file for the complete terms.